# The mod that left

Lab note 12  18 Oct 2026  5 min read
written by Claude from this repository's data

lsp-plus answered Claude's search for a name in your code with the language server's answer: the exact places that use it, from the engine your editor uses. It made the cut in October, when renames on click took 3.4 requests with it instead of 4.6. In 0.7.0 it leaves the pack. This note is why: what its servers cost, how much of the code they actually saw, and two benchmarks that found the pack no more expensive without it.

## What a server costs

A language server is a process per language per session, running from the first question to the session's end. lsp-plus asked the first question itself, at session start, so each server would be warm when Claude needed it.

- **Memory.** pyright on this lab's own repository took 3.5 GB: it read 57,201 Python files, nearly all of them in git-ignored checkouts, where git tracks 82. Told to skip the ignored folders, it took 237 MB. An empty project costs 90 to 130 MB per process, the TypeScript server up to 2 GB on a large repository, and every session starts its own.
- **Without lsp-plus, no server starts.** Claude Code starts a server only when something asks it. In 24 runs where lsp-plus started none (12 without it, 12 with an engine that needs no server), not one diagnostic reached the conversation: no server had started.

## What its answers were worth

- **Who asked.** In 1,931 benchmark runs with the pack, Claude called the language server itself 3 times; cells' `refs()` and `defn()` were called 0 times and `rename()` once in 482 cells. delegate looked up a hand-off's places 412 times in 195 hand-offs, the one real consumer, and it falls back to a text search without a server.
- **What the server saw.** On real code the answer was often missing. On 30 SWE-bench issues in Django and sympy, lsp-plus recognised 55 searches for a code name in 25 runs and answered 6 of them, in 2. We took 128 names from 236 such searches in 29 checkouts. 49% have one definition, 30% several (a median of 5 namesakes), 20% none in the repository. For 49 names with one definition, the share of their uses in the code each engine found, and how many of its hits were real:

| Engine | Found | Precise |
|---|---|---|
| pyright, the language server | 50% | 95% |
| Graphify, a code graph | 49% | 94% |
| Trail-Code, a code graph | 47% | 97% |
| A word search, each hit named by its enclosing function | 100% | 75% |
| The same search, hits sorted into definitions and uses by the files' outlines | 98% | 91% |

The server missed half of the uses because Python here is untyped: `self.query.get_compiler()` names a method on a receiver whose type pyright cannot know. Classes fared worst (36% found), then methods (62%) and functions (68%). The two code graphs, built from the same kind of analysis, missed the same half. A plain word search finds every use and some namesakes, and the files' outlines sort most of the namesakes out in 87 ms, with no server.

## Two benchmarks without it

**On the tasks it was built for** (48 runs, $11.85): a trace and a rename in click, a rename in a TypeScript library (chrono) and a rename in a React app, 3 runs per task, each against the pack as 0.6.1 shipped it:

| Arm | Average task |
|---|---|
| Without lsp-plus | −1.8% [−10.4, +8.1] |
| lsp-plus answering from the word search and outlines, no servers | +3.0% [−8.2, +15.7] |
| lsp-plus with a filter for the server's diagnostics | −4.9% [−12.4, +3.3] |

None of them is clear of zero. The click rename that once shortened by a quarter took 3.3 requests with lsp-plus and 3.7 without, at the same cost: seek, new in 0.6.0, has Claude look around in few, lean steps, and a rename no longer needs the server to be short. The engine without servers saved nothing either: the answer itself was not where the money was.

**On everything else** (72 runs, $34.84): twelve new tasks shaped like the most common Claude Code work, five in a React and TypeScript app, four operating a small service, three in prose and data. Plain Claude Code, the pack as 0.6.1 ships it, and the pack without lsp-plus, 2 runs each:

| | Average task |
|---|---|
| The pack against plain Claude Code | −15.8% [−19.7, −11.9] |
| Without lsp-plus against the pack | −1.6% [−7.2, +4.1] |
| … on the prose and data tasks | −5.2% [−9.2, −1.1] |

Without lsp-plus the pack cost the same, and on prose a little less: a server's start and its reports cost something even where no code is searched.

## The diagnostics

A running server reports problems in the files it has open, and Claude Code adds those reports to the conversation after each tool's result. In one long lab session, 403 reached the conversation; 21% repeated one already sent, moved by a line after an edit; most were about files nobody had edited, from tests the benchmark keeps hidden or from another session's work. A filter that kept only problems in files Claude had edited saved 14.1% [−25.2, −0.7] once what the reports leave in the conversation is priced (note 11). But the reports came from servers lsp-plus had started. Without lsp-plus, there are none to filter.

## What leaves, and what stays

- **Gone with it:** the grep answer, the warm-up, the docs named in references, the check for docs that still name a removed function, and `/refs`.
- **Stays:** delegate and cells use a language server wherever one is installed, and fall back to text search where none is. Nothing in the pack starts one any more.
- **The update.** modpack 0.7.0 no longer installs lsp-plus. At its first start it removes `lsp-plus@claude-modpack` once and says so in one line; someone who installs it again on purpose keeps it. Its last released version, 0.4.2, is in the `v0.6.1` tag.

## What this does not show

- **Small batches.** 2 to 3 runs per task and arm; the intervals are wide. Two batches, 120 runs, point the same way, and neither shows lsp-plus saving anything.
- **Typed code at scale.** The TypeScript tasks are a library and an app of modest size. On a large typed codebase a server sees more of the uses than it did in Django; we have not measured that.
- **What a person does.** Benchmark prompts are complete specifications. A person who asks "who calls this?" many times a day in a typed codebase may get more from a server than these tasks show. The servers are still there for them, through Claude Code's own LSP plugins.

## Method & limits

Arms, with the person's installed plugins switched off (note 8): Z6, the pack as 0.6.1 shipped it; Z6N, without lsp-plus; Z6I, lsp-plus answering from the word search and outlines with no servers; Z6D, lsp-plus with the diagnostics filter; V, plain Claude Code. Claude Opus 5.5, headless, one fresh copy of each repository per run, every run graded: 113 of 120 at full marks, the rest spread over the arms (five on one prose task with strict checks, plain Claude Code included). Cost at list prices; 95% bootstrap intervals over runs within each task. The engine comparison ran each engine over the same checkouts and counted a name's uses against every occurrence of the name in code, outside strings and comments. Runs: `results-engine.jsonl` and `results-beyond.jsonl` in `benchmarks/`; the tasks: `web_repo.py`, `ops_repo.py`, `docs_repo.py`.